

LCTA 演算法簡介：

LCTA 演算法的主要目的是計算隨機網路的完成時間，演算過程會採取一特別的順序，從起始節點開始，逐步拜訪 network 中每一個節點，每個節點在拜訪的過程中，會同步計算起始節點到達該節點的路徑時間，並將相關的計算結果儲存於節點資料結構當中，當完成拜訪後(到達根節點)，隨機網路的完成時間也同時計算完成。

1. LCTA 演算法可透過 ETS 資料結構 $Prec_Node(i)$ 的內容，輕易的將 network 轉換成”樹狀結構”
2. 從”樹狀結構”根節點處開始，採用後序追蹤(Post-Order Tracing)原則拜訪節點樹狀結構(節點拜訪的順序是由左至右)
3. 後序追蹤：參考圖 1 節點 4，根據 $Prec_Node(4) = [1, 2, 3]$ 得知輸入節點共有三個，追蹤會從最左邊節點 1 開始進行，當拜訪完畢會回到節點 4 處，依此類推，再依序拜訪節點 2, 3，最後回到節點 4
4. 追蹤過程中會同步計算節點 1~3 的路徑時間，當回到節點 4 後，便能掌握所有的子節點路徑時間，再執行 max 運算可得到節點 4 的輸出時間
5. 依此追蹤原則及同步計算，面對更複雜的網路結構，便能有順序的拜訪所有的節點，同時也能夠計算每一條路徑時間

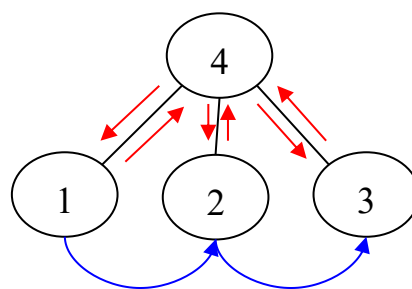


圖 1. 一個簡單樹狀結構

可再參考圖 2 的範例，先將 Network 轉換成樹狀結構，再從根節點 4 開始，可觀察後序追蹤的整體流程：

1. 紅色箭頭往下： $push_stack(node(i))$

2. 紅色箭頭往上：*pull_stack()*
3. 藍色箭頭：拜訪的順序，依序 1→2→3→4
4. 特別注意：節點 1 及節點 2 會被”重複拜訪”，重複拜訪的節點表示已完成計算，必須在節點 3 位置處理重複路徑偏差問題
5. “堆疊”在 LCTA 演算法中扮演非常重要的角色，透過 *push_stack(node(i))*及 *pull_stack()*函式，讓 LCTA 全程掌握節點追蹤順序位置，不會出錯

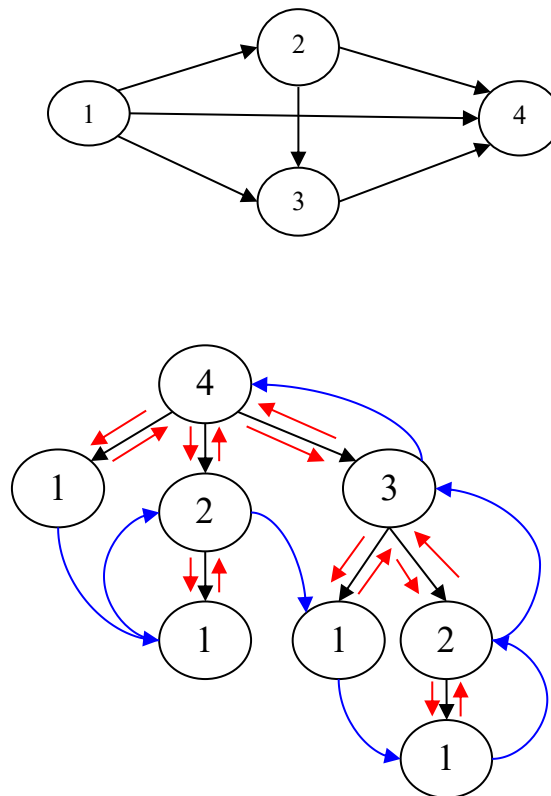


圖 2. 後序追蹤順序說明

數學式定義說明：

- $i \in B_j$ ：表示節點 j 所有的”輸入節點”集合，等同於 $\text{Prec_Node}(j)$
- Y_i^j ：表示 $\text{node}(j)$ 第 i 項輸入節點的”累積路徑時間”，該”累積路徑時間”即為節點 i 的 Output_i 輸出時間
- current_node ：是指追蹤程序正在拜訪的節點標籤
- last_node ：則是上次拜訪完畢的節點標籤

- $\mu(Output_i)$ ：去除變異數計算，將節點 i 的輸出時間去除變異數，最後會形成 $\mu(Output_i)=[E(Output_i), 1]$ 的隨機變數型態， $E(Output_i)$ 是計算 $Output_i$ 的期望值
- *Shared_flag*：負責判斷追蹤過程中，*current_node j* 是否被重複拜訪過(後續有特別的說明)
- $N-1$ ：表示根節點標籤， $finish_flag_N=0$ 表示 Network 尚未完成追蹤運算
- $DRT(Z)$ ：針對 Z 隨機變數進行重新取樣計算
- $\oplus$ ：隨機變數間 convolution 運算
- $\text{Max}\{\dots\}$ ：隨機變數間 max 運算

LCTA 演算法流程：

LCTA 後序追蹤程結合前述 ETS 資料結構可進行節點資料之分析，能夠完成隨機性專案網路完成時間機率分佈之估算。整個演算程序共計兩個演算函式：

1. Upward_Tracing (last_node, current_node)
2. Downward_Tracing (current_node)

Downward_Tracing 函式在演算法中負責往下追蹤，Upward_Tracing 函式則負責往上追蹤，詳細的運作程序描述以列虛擬碼表示：

```

Downward_Tracing (j)           // j = current node
while ( $finish\_flag_{N-1} = 0$ ) do
    if ( $finish\_flag_j = 1$ )       // 節點 j 已完成計算?
         $Shared\_flag = 1$ ;        // 節點 j 完成計算，標註"已被拜訪"
         $k = j$ ;                  // last_node k = j
         $j = pull\_stack()$ ;      // 取出新 current_node 持續往上追蹤
        執行 Upward_Tracing(k, j);
    Else
        if ( $j = 1$ ) // 碰觸起始節點?
             $Output_j = [0, 1]$ ;    // 定義起始節點為隨機變數 0

```

```

     $finish\_flag_j = 1$  ;
     $k = j$ ; // last_node  $k = j$ 
     $j = pull\_stack()$  ; // 取出新 current_node 持續往上追蹤
    執行 Upward_Tracing( $k, j$ ) ;
Else
     $push\_stack(j)$  ; // 將  $j$  至入堆疊中
    由左至右順序，找出節點  $j$  下一個追蹤節點 (根據
    Path_Flag( $j$ ) = { $flag_k$ },  $k \in B_j$ ，若  $flag_k = 0$  則選擇節點  $k$ )
    current_node  $j = i$  ;
End
End
End

Upward_Tracing( $k, j$ ) //  $k = last\_node$ ,  $j = current\_node$ 
while ( $finish\_flag_{N-1} = 0$ ) do
    if ( $Shared\_flag = 1$ ) // 表示 last_node  $k$  已經被拜訪過
         $Y_k^j = \mu(Output_k)$ ; // 去除變異數
         $Shared\_flag = 0$  ; // Reset  $Shared\_flag$  value
    Else
         $Y_k^j = Output_k$ ;
        Path_Flag( $j$ ) = { $flag_k = 1$ },  $k \in B_j$  ;
    End
    if ( $Y_k^j$  的樣本單元大於 100)
         $DRT(Y_k^j)$  ; // 執行重新取樣計算工作
    End
    if ( $finish\_flag_j = 0$ ) // 節點  $j$  尚未完成計算?
         $push\_stack(j)$  ; // 將 current_node  $j$  至入堆疊中
        由左至右順序，找出節點  $j$  下一個追蹤節點 (根據
        Path_Flag( $j$ ) = { $flag_k$ },  $k \in B_j$ ，若  $flag_k = 0$  則選擇節點  $k$ )
        current_node  $j = i$  ;
        執行 Downward_Tracing ( $j$ ) ;
    Else // 完成節點  $j$  的計算
         $Output_j = \max\{Y_i^j\} \oplus Node\_Time(j)$ ,  $i \in B_j$  ;

```

```

    finish_flagj = 1 ;
    k = j;                // last_node k = j
    j = pull_stack() ;    // 取出新 current_node 持續往上追蹤
End
End

```

LCTA 演算法說明：

LCTA 演算會從根節點 n 開始進行追蹤，會先執行 Downward_Tracing(n) 函式，演算過程中 Upward_Tracing(k, j) 及 Downward_Tracing(j) 兩個函式會相互交替進行，過程中資料計算的結果會紀錄在 ETS 的節點資料結構當中。

演算採後序追蹤原則，樹狀結構節點拜訪的順序是由左至右，因此追蹤路線會沿左側邊一直下探，下探的過程會執行 *push_stack(j)* 指令，將所經過的節點紀錄在堆疊中，以便紀錄追蹤之路徑。

當追蹤程序碰觸起始節點 1，便開始啟動 Upward_Tracing(k, j) 追蹤程序，透過 *pull_stack()* 指令尋求上一層父節點標籤，沿著先前紀錄路徑往上爬升，過程中也開始執行路徑時間累加計算，計算的結果也會紀錄在節點資料結構中。

LCTA 透過 *push_stack(j)*、*pull_stack()* 指令，存取執行節點的標籤，能夠有序的讓後序追蹤演算順利進行

Shared_flag 旗標特別說明：

該旗標是負責判斷追蹤過程中 current_node j 是否被重複拜訪過

Upward_Tracing 函式中，當節點 j 完成計算後(所有的輸入路徑時間均已完成計算)，仍持續往上追蹤，此時節點 j 雖已完成拜訪，但仍屬首次，故 *Shared_flag*=0。

Downward_Tracing 函式中，若再度遇到已完成計算的節點 j ，便屬於”重複拜訪”，因此 *Shared_flag*=1。

追蹤過程中當 current_node = j 且 *Shared_flag*=1 當下，表示重複拜訪節點事件發生，也表示”路徑相依性問題”發生！此時的 LCTA 演算

法就必須要處理該問題，已置於演算法程序當中。